8. Special / Miscellaneous Operators

- A part from these operators, there are a few operators that do not fit in any of the above categories. These are:-

1. sizeof Operator

- Return the size (in byte) of a data type or variable.
- Syntax : sizeof(data_type / Variable_name)
- Example:-

```c
#include<stdio.h> // printf() scanf()
#include<conio.h> // getch()
#include<stdbool.h> // bool
int main()
{
int a=10;
printf("size of int = %d byte \n",sizeof(int));
printf("size of char = %d byte \n",sizeof(char));
printf("size of long int = %d byte \n",sizeof(long long int));
printf("size of float = %d byte \n",sizeof(float));
```

```
C:\Users\hira\Desktop\hello.exe
size of int = 4 byte
size of char = 1 byte
size of long int = 8 byte
size of float = 4 byte
```

2. Comma Operator(,)

- Evaluates multiple expressions and returns the value of the last expression.
- Example : int a=5,b=10;

  int c = (a++,b++,a+b);

  printf("value of c = %d",c);   // c = 17

3. Address-of-operator (&)

- Return the memory address of a variable.

- Example :
  Int a=10;
  Printf(“address of a = %d”,&a);   //ask address of operator

```c
int main()
{
int a=10;
printf("address of a = %d \n",&a);
printf("value of a = %d \n",a);
```

```
C:\Users\hira\Desktop\hello.exe
address of a = 450885500
value of a = 10
```

4. Derefernce Operator (*)

- Accesses the value at a memory address (opposite of &)

```c
int main()
{
int a=10;
printf("value of a = %d \n",a);
// print value of a using pointer (2nd method)
int *ptr=&a;
printf("value of a = %d",*ptr);
```

```
C:\Users\hira\Desktop\hello.exe
value of a = 10
value of a = 10
```

5. Dot operator (.)

- Accesses members of an object/struct.

```cpp
#include<iostream>
#include<conio.h>
using namespace std;
struct student
{
    string name;
    int age;
};
int main()
 {
     student s1;
     s1.name ="Hira"; // dot operator
     s1.age = 90;
     cout << "Name = " << s1.name << " & Age = " << s1.age;
      getch();
return 0;
}
```

C:\Users\hira\Desktop\hira.exe

```
Name = Hira & Age = 90
```

<table>
<tr><th>Operator Name</th><th>Operator</th><th>Description</th><th>Associativity</th></tr>
<tr><td>Partentheses Operator</td><td>()
[]
.
->
++ --</td><td>Parentheses or function call
Brackets or array subscript
Dot or Member selection operator
Arrow operator
Postfix increment/decrement</td><td>left to right</td></tr>
<tr><td>Unary Operator</td><td>++ --
+ -
! ~
(type)
*
&
sizeof</td><td>Prefix increment/decrement
Unary plus and minus
not operator and bitwise complement
type cast
Indirection or dereference operator
Address of operator
Determine size in bytes</td><td>right to left</td></tr>
<tr><td>Multiplication Operator</td><td>* / %</td><td>Multiplication, division and modulus</td><td>left to right</td></tr>
<tr><td>Addition Operator</td><td>+ -</td><td>Addition and subtraction</td><td>left to right</td></tr>
<tr><td>shift operators</td><td><< >></td><td>Bitwise left shift and right shift</td><td>left to right</td></tr>
<tr><td rowspan="2">Relational Operator</td><td>< <=
> >=</td><td>relational less than/less than equal to
relational greater than/greater than or equal to</td><td>left to right</td></tr>
<tr><td>== !=</td><td>Relational equal to or not equal to</td><td>left to right</td></tr>
<tr><td rowspan="3">Bitwise Operator</td><td>&&</td><td>Bitwise AND</td><td>left to right</td></tr>
<tr><td>^</td><td>Bitwise exclusive OR</td><td>left to right</td></tr>
<tr><td>|</td><td>Bitwise inclusive OR</td><td>left to right</td></tr>
<tr><td rowspan="2">Logical Operator</td><td>&&</td><td>Logical AND</td><td>left to right</td></tr>
<tr><td>||</td><td>Logical OR</td><td>left to right</td></tr>
<tr><td>Conditional / Ternary Operator</td><td>? :</td><td>Ternary operator</td><td>right to left</td></tr>
<tr><td>Assignment Operator</td><td>=
+= -=
*= /=
%= &=
^= |=
<<= >>=</td><td>Assignment operator
Addition/subtraction assignment
Multiplication/division assignment
Modulus and bitwise assignment
Bitwise exclusive/inclusive OR assignment</td><td>right to left</td></tr>
<tr><td>Comma Operator</td><td>,</td><td>comma operator</td><td>left to right</td></tr>
</table>

Precedence(Priority)

PUMAS REBL TAC
1 2 3 4 5 6789 101112

solve priority number

Just like BODMAS

TAU (Right to left)
T- Ternary
A-Assignment
U-Unary operator

=====================HIRA KUMAR=======================

HIRA KUMAR